# Diplomarbeit

## Nextmove

Interaktives Schachbrett

Eingereicht von

**Maximilian Koch**
**Christoph Schwnaninger**
**Christina Seidner**

Eingereicht bei

**Höhere Technische Bundeslehr- und Versuchsanstalt Anichstraße**

Abteilung Elektronik und technische Informatik

Betreuer

Schiechtl Kevin, Dipl. -Ing. (FH)
Götsch Leopold, DI

Innsbruck, März 2026

---

Abgabevermerk: Betreuer/in:

Datum:

# Kurzfassung /Abstract

Diese Diplomarbeit beschäftigt sich mit der Entwicklung eines interaktiven, einsteigerfreundlichen Schachbretts, das ohne jegliche Vorkenntnisse den Einstieg in das Schachspiel ermöglicht. Das System erkennt die vom Spieler ausgeführten Züge, zeigt zulässige Zugmöglichkeiten visuell per Lichtsignal auf dem Spielbrett an und unterstützt den Spieler durch eine Spielanalyse auf einer Website, welche auf fehlerhafte oder ungünstige Züge hinweist. Ziel des Projekts ist es, eine intuitive Lernhilfe zu schaffen, die insbesondere Anfängern dabei hilft, die Spielregeln zu verstehen und ihre Fähigkeiten schrittweise zu verbessern.

Das Konzept basiert auf einer eigens entwickelten Hardware in Kombination mit einer darauf abgestimmten Softwarelösung. Ein zentraler Bestandteil der Arbeit ist die Konstruktion und das Design des Schachbretts sowie die Programmierung der Steuer- und Auswerteeinheiten. Die lokale Verarbeitung der Spieldaten ermöglicht eine direkte Rückmeldung während des Spiels. Ergänzend dazu wurde eine Serveranbindung realisiert, über die Spielstände und Analysedaten gespeichert werden.

Trotz auftretender Herausforderungen bei der Hardwareintegration und der Kommunikation zwischen lokaler Einheit und Server konnte ein funktionales Gesamtkonzept umgesetzt werden. Zukünftige Weiterentwicklungen könnten eine verbesserte Fehlererkennung, eine erweiterte Spielanalyse sowie eine optimierte Benutzeroberfläche umfassen. Insgesamt bildet diese Arbeit die Grundlage für ein digitales Schachsystem, das moderne Technik nutzt, um den Zugang zum Schachspiel nachhaltig zu erleichtern und neue Möglichkeiten für das spielerische Lernen zu eröffnen.

This thesis deals with the development of an interactive, beginner-friendly chessboard that allows players to start playing chess without any prior knowledge. The system recognizes the moves made by the player, visually displays possible moves on the game board using light signals, and supports the player by analyzing the game on a website, which points out incorrect or unfavorable moves. The aim of the project is to create an intuitive learning aid that helps beginners in particular to understand the rules of the game and gradually improve their skills.

The concept is based on specially developed hardware in combination with a customized software solution. A central component of the work is the construction and design of the chessboard as well as the programming of the control and evaluation units. Local processing of the game data enables direct feedback during the game. In addition, a server connection has been implemented to store game scores and analysis data.

Despite challenges encountered in hardware integration and communication between the local unit and the server, a functional overall concept was implemented. Future developments could include improved error detection, enhanced game analysis, and an optimized user interface. Overall, this work forms the basis for a digital chess system that uses modern technology to make chess more accessible in the long term and open up new opportunities for playful learning.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

# Erklärung der Eigenständigkeit der Arbeit

**EIDESSTATTLICHE ERKLÄRUNG**

Ich erkläre an Eides statt, dass ich die vorliegende Arbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe. Meine Arbeit darf öffentlich zugänglich gemacht werden, wenn kein Sperrvermerk vorliegt.

Ort, Datum Schwaninger Christoph

Ort, Datum Seidner Christina

Ort, Datum Koch Maximilian

# Inhaltsverzeichnis

**Abstract** ii

**1 Einleitung** 1
1.1 Vertiefende Aufgabenstellung . . . . . 2
1.1.1 Schwaninger Christoph . . . . . 2
1.1.2 Koch Maximilian . . . . . 3
1.1.3 Seidner Christina . . . . . 4
1.2 Dokumentation der Arbeit . . . . . 5

**2 Lösungsidee** 7
2.1 Hardware . . . . . 7
2.1.1 Bauteilauswahl . . . . . 7
2.1.2 Schaltungsentwicklung . . . . . 8
2.1.3 Platinendesign - PCB . . . . . 8
2.1.4 Gehäusefertigung . . . . . 8
2.1.5 Stromversorgung . . . . . 9
2.2 Software . . . . . 9
2.2.1 Magneterkennung . . . . . 9
2.2.2 Matrixauslesung . . . . . 9
2.2.3 Entwicklung eines Menüs . . . . . 9
2.2.4 Spiellogik . . . . . 9
2.2.5 Einbindung des Spielererkennung . . . . . 9
2.3 Webinterface . . . . . 10
2.3.1 Auswahl der Komponenten . . . . . 10
2.3.2 Website . . . . . 11
2.3.3 Datenbank . . . . . 11
2.3.4 Server . . . . . 11

**3 Lösung** **13**
3.1 Hardware . . . 13
3.1.1 Bauteilauswahl . . . 13
3.1.2 Schaltungsentwicklung . . . 17
3.1.3 Platinendesign . . . 21
3.1.4 Gehäusefertigung . . . 21
3.1.5 Stromversorgung . . . 21
3.2 Software . . . 21
3.2.1 Magneterkennung . . . 21
3.2.2 Matrixauslesung . . . 21
3.2.3 Entwicklung eines Menüs . . . 21
3.2.4 Spiellogik . . . 21
3.2.5 Einbindung des Spielererkennung . . . 21
3.3 Webinterface . . . 22
3.3.1 Website . . . 22
3.3.2 Datenbank . . . 22
3.3.3 Server . . . 22

**4 Latex-Beispiele** **23**
4.1 Auflistungen . . . 23
4.2 Tabellen . . . 24
4.3 Abbildungen . . . 25
4.4 Quellcode . . . 25
4.4.1 Java-Code . . . 25
4.4.2 Python-Code . . . 26
4.4.3 Lesen von Dateien . . . 26
4.5 Referenzen . . . 27
4.6 Zitate . . . 27

**Literaturverzeichnis** **37**

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

# 1 Einleitung

Schach ist ein traditionsreiches Brettspiel, welches logisches Denken und Konzentration fördert. Durch den zunehmenden Einsatz digitaler Medien werden klassische Spiele jedoch immer häufiger durch rein elektronische Lösungen ersetzt. Aus diesem Grund entstand die Idee, ein herkömmliches Schachbrett mit moderner Technik zu erweitern und dadurch ein interaktives Spielerlebnis zu schaffen.

Diese Diplomarbeit befasst sich mit der Entwicklung eines interaktiven, smarten Schachbretts, das klassische Spielmechanismen mit moderner Technik verbindet. Durch die Kombination aus Sensorik, lokaler und externer Datenverarbeitung mittels passender Software soll ein System entstehen, das Spielzüge automatisch erkennt, den Spieler aktiv unterstützt und nach der Partie zeigt welche Züge besser geeignet gewesen wären. Dadurch wird das Schachspiel sowohl für Anfänger als auch für erfahrene Spieler attraktiver gestaltet.

Die Aufgabenstellung umfasst die Planung und Umsetzung der elektronischen und softwaretechnischen Komponenten sowie deren Integration in ein funktionales Gesamtsystem. Das Projekt verbindet technische Fachbereiche wie die Entwicklung und Programmierung der benötigten Hardware und Informationsaustausch mit einem speziellen Server auf dem der Nutzer seine Partien analysieren kann.

## 1.1 Vertiefende Aufgabenstellung

### 1.1.1 Schwaninger Christoph

#### 1.1.1.1 Bauteilauswahl

Auswahl aller Hardware-Komponenten, welche für die Umsetzung des intelligenten Schachbretts benötigt werden.

#### 1.1.1.2 Schaltungsentwicklung

Entwicklung einer Verschaltungslogik welche die Ansteuerung und Informationsausgabe der Felder effizient ermöglicht.

#### 1.1.1.3 Platinendesign

Die Zuvor entwickelte Verschaltung der Komponenten für einen Platinenfertiger nach Anforderung erstellen.

#### 1.1.1.4 Gehäusefertigung

Design, assemblieren und Einbindung aller benötigten Komponenten in das Gehäuse.

#### 1.1.1.5 Stromversorgung

Auswahl und Verkabelung eines Stromversorgungssystem, welche den Anforderung aller Hardware-Komponenten entspricht.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

## 1.1.2 Koch Maximilian

### 1.1.2.1 Magneterkennung

Bei näherung eines Magneten an einen Sensor, wird dessen dasein vom Controller erkannt.

### 1.1.2.2 Matrixauslesung

Anhand einer vorher selbstgebauten Platine die Position von Magneten in einer Matrix feststellen.

### 1.1.2.3 Entwicklung eines Menüs

Entwicklung eines Menüs auf dem OLED-Bildschirm des Schachbretts. Eine Auswahl zwischen Spieler gegen Spieler oder Spieler gegen Computer erstellen.

### 1.1.2.4 Spiellogik

**1.1.2.4.1 Das Einzelspiel (Spieler gegen Computer)** Erstellen einer Spiellogik, mit der man gegen einen Schachcomputer spielen kann. Durch das Auswählen zwischen drei Schwierigkeitsstufen, kann der Spieler extra gefordert werden. Am Ende kann das Spiel auf der Website angeschaut werden.

**1.1.2.4.2 Der Mehrspieler (Spieler gegen Spieler)** Erstellen einer Spiellogik, mit der ein Spieler gegen einen anderen spielen kann. Am Ende kann das Spiel auf der Website angeschaut werden.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

#### 1.1.2.5 Einbindung der Spielererkennung

Durch das Auslesen des eingebauten Fingerabdrucksensors die Unterscheidung verschiedener Spieler sicherstellen.

### 1.1.3 Seidner Christina

#### 1.1.3.1 Die Website

Auf der Website können sich Spieler mithilfe des NFC-Sensors am Schachbrett registrieren, um dann ihre gespielten Spiele zu analysieren. Zudem kann man auf der Website Schach spielen.

#### 1.1.3.2 Die Datenbank

Auf der Datenbank werden alle relevanten Daten gespeichert, wie beispielsweise Benutzername, Rückgabewert des NFC-Sensors und die Schachzüge.

#### 1.1.3.3 Der Server

Der Server verbindet Website, Datenbank und Schachbrett miteinander.

## 1.2 Dokumentation der Arbeit

Es werden die Projektergebnisse dokumentiert

- Grundkonzept
- Theoretische Grundlagen
- Praktische Umsetzung
- Lösungsweg
- Alternativer Lösungsweg
- Ergebnisse inkl. Interpretation

Weitere Anregungen:

- Fertigungsunterlagen
- Testfälle (Messergebnisse...)
- Benutzerdokumentation
- Verwendete Technologien und Entwicklungswerkzeuge

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

# 2 Lösungsidee

## 2.1 Hardware

### 2.1.1 Bauteilauswahl

#### 2.1.1.1 Computer

Bei der Auswahl des Rechners für das Schachbrett ist entscheidend, dass dieser über ausreichende Rechenleistung zur zuverlässigen Verarbeitung der Sensordaten verfügt und gleichzeitig energieeffizient arbeitet. Zudem sollte er kompakt, kostengünstig und gut in das Gesamtsystem integrierbar sein. Ein Rechner welcher mittels Python programmierbar ist, wäre für Maximilian Koch die bevorzugte Lösung.

#### 2.1.1.2 Sensoren

Bei der Auswahl der Sensoren für das Schachbrett stellen Kosten, Zuverlässigkeit und Praktikabilität die drei Hauptauswahlkriterien dar, anhand derer die zur Verfügung stehenden Methoden bewertet wurden. Die Kosten sind unter anderem ein wichtiges Auswahlkriterium, da für das Schachbrett insgesamt 64 Einheiten benötigt werden. Die Zuverlässigkeit ist ein weiterer zentraler Punkt, da fehlerhafte Erkennungen zu falschen Spielzuständen und somit zu einem fehlerhaften Spielablauf führen können. Als letztes Kriterium wird die Praktikabilität betrachtet, da die gewählte Methode die optische Anzeige des Schachbretts möglichst wenig bis gar nicht einschränken darf, um ein gutes Spielerlebnis zu gewährleisten.

### 2.1.2 Schaltungsentwicklung

Bei der Schaltungsentwicklung geht es darum die 64 Einheiten effizient zu verschalten, um die Anzahl der benötigten Pins zu verringern, da auf dem Rechner keine 64 einzelnen Pins zur Verfügung stehen werden. Zusätzlich sollen die Signale auch das richtige Logiklevel, für den verwendedten Microcontroller oder Einplatinenpc, haben. Möglichkeiten die Pins zu reduzieren wäre eine Zeilen-Spalten Matrix, welche die benötigten Pins von 64 auf 16 begrenzen würde. Diese Lösung würde einen guten Kompromiss zwischen technischer Komplexität und Programmierbarkeit der Hardware ermöglichen.

### 2.1.3 Platinendesign - PCB

Das Platine muss so designed werden, dass Anschlüsse für die Recheneinheit, Sensorik und elektronische Bauteile ihren Zweck erfüllen und physisch an der richtigen Stelle sind. Die Anschlüsse sollen über der Recheneinheit platziert werden um lange Verbindugnsleitung zu verhindern. Die Sensoren, für die Figuren Erkennung, müssen sich genau an den zuvor berechneten Stellen befinden, damit beim zusammenbau von Gehäuse und Elektronik die Sensor Positionen mit den Feldposition übereinstimmen.

### 2.1.4 Gehäusefertigung

Bei der Gehäusefertigung sollen folgende Punkte erfüllt werden:

- Einbindung der Bedienelemente
- Kühlung und Lüftungskonzept
- Handlichkeit
- technischen Elemente der Spiellogik vor dem Nutzer verstecken
- sichere Befestigungen für die internen Bauteile

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

### 2.1.5 Stromversorgung

Das System, für die Stromversorgung, soll es dem Nutzer einfach machen das Schachbrett zu nutzen. Ein USB-C Ladeanschluss über den der Nutzer ohne technisches Verständnis das Schachbrett nutzen kann wurde dafür ausgewählt. Für die technische Umsetzung muss bei dieser Methode jedoch eine Ladelogik für einen internen Akku und Umschaltlogik zwischen Akku und Netzteil-Betrieb entwickelt werden.

## 2.2 Software

### 2.2.1 Magneterkennung

### 2.2.2 Matrixauslesung

### 2.2.3 Entwicklung eines Menüs

### 2.2.4 Spiellogik

#### 2.2.4.1 Das Einzelspiel (Spieler gegen Computer)

#### 2.2.4.2 Der Mehrspieler (Spieler gegen Spieler)

### 2.2.5 Einbindung des Spielererkennung

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

## 2.3 Webinterface

### 2.3.1 Auswahl der Komponenten

#### 2.3.1.1 Programmiersprachen

Es stand am Anfang die Wahl zwischen PyScript und JavaScript. PyScript ist ein Tool, um Python in eine Web-Anwendung einzubinden, jedoch steht es noch in den Startlöchern und ist damit noch zu unprofessionell und unzuverlässig. Wäre PyScript weiter entwickelt , wäre es eine interessante Alternative zu JavaScript gewesen, da Python schon in den letzten Jahren gelernt wurde und JavaScript nicht.

#### 2.3.1.2 WebTools für den Server

Hier steht die Auswahl zwischen den Webservern Nginx und Apache im Vordergrund. Nginx ist die modernere und leistungsfähigere Variante von den beiden. Auch wenn momentan noch nicht viele Spieler gleichzeitig spielen, kann das in Zukunft vorkommen. Dafür ist ein leistungsfähiger und schneller Webserver wie Nginx essenziell. Dazu wurde der Anwendungsserver uWSGi verwendet, da er sehr gut mit Nginx und Flask zusammenarbeitet.

Das Web-Framework Flask wurde gewählt, da es fast leer startet und man selbst wählen kann, welche Bibliotheken man nutzt. Das führt dazu, dass weniger Arbeitsspeicher verwendet wird, was auf einem Debian-Server wichtig ist. Weiters braucht man durch die mit HTML und JavaScript geschriebene Website nur eine API, die JSON-Daten sendet und empfängt, und Flask ist genau darauf spezialisiert. Außerdem ist Flask sehr ähnlich zu Python, weshalb es leicht verständlich ist, es ist egal welche Datenbank man wählt und es arbeitet sehr nahe mit uWSGI und Nginx zusammen.

#### 2.3.1.3 Server

Es wurde ein Debian Server von der Schule verwendet. Er wird mit ssh konfiguriert.

#### 2.3.1.4 Datenbank

Die Wahl zwischen MariaDB und MySQL war nicht schwer. Das liegt daran, dass MariaDB eine modifizierte und damit modernere Version von MySQL ist.

### 2.3.2 Website

Das Ziel der Website ist es:

- einen Benutzer anlegen zu können,
- online Schach spielen zu können
- und die bereits gespielten Spiele analysieren zu können

### 2.3.3 Datenbank

Die Datenbank speichert alle wichtigen Daten. Das sind die Benutzernamen mit Ausgabe des NFC-Sensors, Schachzüge und Spiele.

### 2.3.4 Server

Der Server verbindet die Website, Datenbank und den Raspberry Pi des Schachbretts miteinander und sorgt für eine reibungsfreie Kommunikation.

# 3 Lösung

## 3.1 Hardware

### 3.1.1 Bauteilauswahl

#### 3.1.1.1 Computer

Für die Diplomarbeit wurden mehrere Rechner in betracht gezogen, jedoch wurde die Entscheidung zwischen den folgenden beiden getroffen aus dem Grund, dass beide sofort zur Verfügung standen:

- **Raspberry Pi 5:** Der Raspberry pi 5 verfügt über einen Quad-Core ARM Cortex-A76 Prozessor wodurch genug Rechenleistung bereitgestellt ist. Zusätzlich bietet dieser genug Schnittstellen um mit einer einfach 8x8 Matrix die Schaltung zu entwickeln. Ein weiterer Punkt ist die gute Dokumentation, möglichen Librarys und die Programmiermöglichkeit mittels Pyhton welche, für Diplomant Maximilian Koch, bevorzugt ist. Die Nachteile des Raspberry Pi sind zum einen den bnötigten Platz welchen dieser benötigt, die Anschaffungskosten für einen und der höhere Stromverbrauch.

- **ESP32-S3:** Der ESP32-S3 verfügt über einen Dual-Core-Xtensa-LX7 Prozsessor welcher für das auslesen der Matrix genug Rechenleistung bereitstellen könnte. Für den ESP32-S3 spricht auch seine kompakte Größe, seine geringe Leisutngsaufnahme und die geringen Anschaffungskosten. Die Nachteile des ESP32 sind zum einen aber auch die begrenzte Rechenleistung. Die weitere Hardware welche auch vom ESP gesteuert wreden muss, könnte den ESP schnell an seine Grenzen

bringen und das fehlen eines vollwertigem Betriebssystem welches das Umsetzen von komplexeren Softwarestrukturen erschwert.

Die Entscheidung fiel auf den Raspberry Pi 5, da dieser bereits verfügbar war und über ausreichende Rechenleistung für das Projekt verfügt. Zudem stellte die Möglichkeit der Programmierung in Python, welche von Diplomant Maximilian Koch bevorzugt wird, einen weiteren ausschlaggebenden Faktor dar, wobei grundsätzlich auch andere Raspberry Pi Modelle für dieses Projekt geeignet gewesen wären.

#### 3.1.1.2 Sensoren

Bei der Auswahl der Sensoren, die zur Erkennung der Spielfiguren sind, wurden mehrere Optionen in betracht gezogen:

- **Hall-Sensoren:**

| **Merkmal** | **Beschreibung** |
|---|---|
| Technologie | Digitaler Hall-Effekt-Sensor |
| Beispielbauteil | A3144 (Digitaler Sensor) |
| Betriebsspannung | 4,5 V – 24 V QUELLE!!! |
| Ausgang | Digital (Open-Collector) |
| Funktionsprinzip | Erkennung eines Magnetfeldes durch Hall-Effekt |
| Vorteile | Kontaktlos, verschleißfrei, hohe Lebensdauer, unempfindlich gegen Staub und Licht |
| Nachteile | Magnet in jeder Figur erforderlich |
| Kostenbewertung | Gering |
| Zuverlässigkeit | Sehr hoch, geringe Fehleranfälligkeit |
| Praktikabilität | Sehr hoch, klein, einfache Beschaltung |

Tabelle 3.1: Bewertung der Hall-Sensoren zur Figurenerkennung

- **NFC/RFID:**

| **Merkmal** | **Beschreibung** |
|---|---|
| Technologie | NFC / HF-RFID |
| Beispielbauteile | PN532 oder MFRC522 (Reader), NTAG213 (Tag) |
| Betriebsspannung | Typ. 3,3 V – 5 V (modulabhängig) |
| Ausgang / Schnittstelle | I²C / SPI / UART |
| Funktionsprinzip | Kontaktlose Identifikation einer Figur über eindeutige Tag-ID |
| Vorteile | Eindeutige Figurenidentifikation möglich, kontaktlos |
| Nachteile | Höherer Hardware- und Softwareaufwand, Metalle können Reader beeinflussen |
| Kostenbewertung | Mittel bis hoch |
| Zuverlässigkeit | Sehr hoch |
| Praktikabilität | Mittel; Integration auf 64 Feldern erfordert Multiplexing |

Tabelle 3.2: Bewertung von NFC/RFID zur Figurenerkennung

- **Optische Sensoren:**

| **Merkmal** | **Beschreibung** |
|---|---|
| Technologie | Infrarot-Reflexionssensor |
| Beispielbauteile | TCRT5000 oder CNY70 |
| Betriebsspannung | Typ. 5 V |
| Ausgang / Schnittstelle | Analog oder digital (je nach Beschaltung) |
| Funktionsprinzip | IR-LED sendet Licht; Phototransistor misst reflektiertes Signal |
| Vorteile | Keine Magnete oder Tags erforderlich, direkte Objekterkennung |
| Nachteile | Empfindlich gegenüber Fremdlicht und Verschmutzung |
| Kostenbewertung | Mittel |
| Zuverlässigkeit | Mittel; abhängig von Lichtverhältnissen, Oberflächenbeschaffenheit und Kalibrierung. |
| Praktikabilität | Mittel; exakte Ausrichtung und ggf. Kalibrierung pro Feld erforderlich. |

Tabelle 3.3: Bewertung optischer Sensoren zur Figurenerkennung

Nach dem Vergleich der untersuchten Sensortechnologien wurde der digitale Hall-Sensor als bevorzugte Lösung ausgewählt. Er bietet eine sehr hohe Zuverlässigkeit durch kontaktlose und verschleißfreie Arbeitsweise bei gleichzeitig geringem Kosten- und Integrationsaufwand.

Optische Sensoren weisen eine erhöhte Empfindlichkeit gegenüber Umgebungslicht, Verschmutzung und Kalibrierungsabweichungen auf, wodurch die Langzeitstabilität beeinträchtigt werden kann. NFC/RFID-Systeme ermöglichen zwar eine eindeutige Identifikation jeder Figur, verursachen jedoch einen deutlich höheren Hardwareaufwand, insbesondere bei der Umsetzung auf 64 Spielfeldern.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

Unter Berücksichtigung der Anforderungen an Kosten, Zuverlässigkeit, Integrationsaufwand und Systemkomplexität stellt der Hall-Sensor die technisch und wirtschaftlich sinnvollste Lösung für das elektronische Schachbrett dar.

### 3.1.2 Schaltungsentwicklung

Bei der Entwicklung der Verschaltungslogik muss damit begonnen werden, wie ein einzelner Sensor korrekt ohne Fehler ausgelesen werden kann. Wichtig ist auch, dass der Raspberry pi mit einem Logikpegel von 3,3V arbeitet und die Hall-Sensoren laut Datenblatt eine Spannung von mindestens 4,5V benötigen. Falls die Sensoren nicht mit 3,3V Versorgungsspannung funktionieren muss hier ein Logic-Level-Shifter verwedent werden

Beschaltung des A3144:

Der Hall-Sensor verfügt über jeweils einen VCC, GND und OUT pin. Der Sensor schaltet den Ausgang nur dann auf LOW, (Open-Collector) wenn der Südpol von einem Magneten in Reichweite ist.

Die Ausgänge vom Sensor und Verschaltung dieser sehen wie folgt aus:

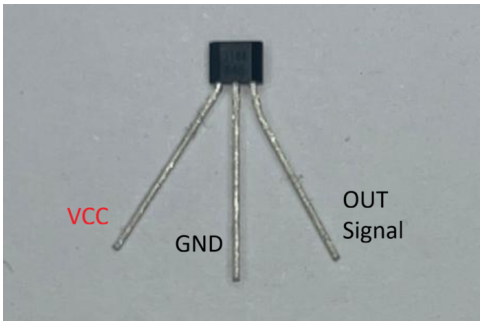


Abbildung 3.1: Nahaufnahme des A3144

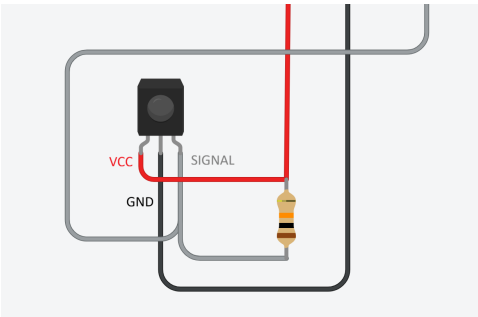


Abbildung 3.2: Verschaltung des A3144

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

#### 3.1.2.1 Pull-Up Widerstandsberechnung

Der A3144 besitzt einen Open-Collector-Ausgang, weshalb ein externer Pull-Up-Widerstand erforderlich ist, um im HIGH-Zustand eine definierte Ausgangsspannung zu erzeugen. Die Dimensionierung erfolgt auf Grundlage des Ohmschen Gesetzes. Der Widerstand bestimmt dabei den Strom im LOW-Zustand sowie die Schaltgeschwindigkeit des Signals. Die Berechnung erfolgt mit

$$R_{PU} = \frac{V_{CC} - V_{CE(sat)}}{I_{LOW}}$$

wobei $V_{CC}$ die Versorgungsspannung, $V_{CE(sat)}$ die Sättigungsspannung des Transistors (Datenblatt $0{,}4\,V$) und $I_{LOW}$ der gewünschte Strom im eingeschalteten Zustand ist. Für $V_{CC} = 3{,}3\,V$ und $I_{LOW} = 1\,mA$ ergibt sich

$$R_{PU} = \frac{3{,}3\,V - 0{,}2\,V}{0{,}001\,A} = 2{,}9\,k\Omega$$

Als geeigneter Normwert wird $3{,}3\,k\Omega$ gewählt. Ein kleinerer Widerstand erhöht den Strom und verbessert die Schaltgeschwindigkeit, während ein größerer Widerstand den Stromverbrauch reduziert, jedoch Slangsamere Signalflanken bringt.

Die Schaltgeschwindigkeit hängt vom Pull-Up-Widerstand ab, da dieser zusammen mit der parasitären Leitungskapazität $C$ eine RC-Zeitkonstante bildet. Beim Übergang von LOW nach HIGH wird die Leitung über den Widerstand aufgeladen, wobei die Anstiegszeit durch die Zeitkonstante

$$\tau = R \cdot C$$

bestimmt wird. Die Spannung steigt dabei exponentiell nach

$$U(t) = U_{CC}\left(1 - e^{-\frac{t}{RC}}\right)$$

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

an. Ein größerer Widerstand führt zu einer größeren Zeitkonstante $\tau$ und somit zu einem langsameren Spannungsanstieg, während ein kleinerer Widerstand eine kleinere Zeitkonstante und damit eine schnellere Schaltflanke bewirkt.

#### 3.1.2.2 Schaltungsaufbau

Um die korrekte Verdrahtung zu testen wurde von Diplomant Maximilian Koch ein Pyhton Programm geschrieben welches unter "Bildverwis" zu sehen ist. Der Output von diesem Programm ist unter "Bildverweis" zu sehen.

Aufbau auf einem Steckbrett zur Inbetriebnahme der Sensoren:

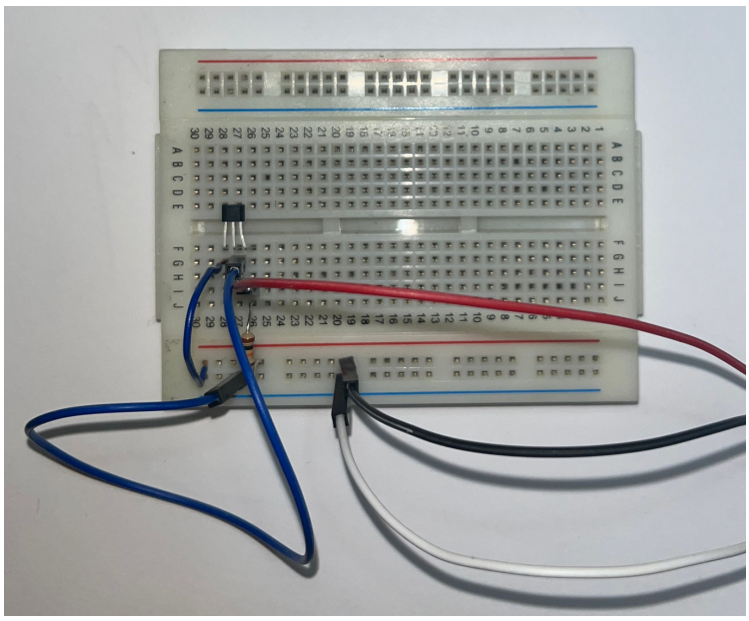

Abbildung 3.3: Einzelbeschaltung

rotes Jumper-Kabel -> Signal
blaues Jumper Kabel mittig -> GND
blaues Jumper Kabel rechts -> VCC

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

#### 3.1.2.3 Matrixentwicklung

Wichtige Punkte zu beachten für die Matrix sind:

- Leistungsaufnahme der Sensoren
- Schaltzeiten um Fehler zu verhindern
- Störsicherheit

**3.1.2.3.1 Leistungsaufnahme** Der gewählte A3144 Hall-Effekt Sensor benötigt typischerweise 4,4mA bis zu maximal 9 mA "Quelle". Bei acht parallel angeschlossenen Sensoren ergibt sich so ein Strombedarf von ca. 35 mA bis maximal 72 mA. Ein GPIO Pin des Raspberry Pi ist allerdings nur für ungefähr 16 mA fähig "Quelle". Deshalb wird für die Versorgung der Sensoren ein MOSFET als Schalter benötigt.

**3.1.2.3.2 Schaltzeiten** Um für zuverlässige Auslesungen der Matrix zu sorgen müssen Anstiegs und Abfallzeiten der Signalleitungen als auch die Schaltzeiten der MOSFETS beachtet werden um überlappende Signale zu verhindern. Dafür wird gesorgt indem Diplomant Maximilian Koch ein entsprechendes Delay im Code zur Auslesung der Matrix implementiert.

**3.1.2.3.3 Störsicherheit** Die Matrix sollte so konzipiert und umgesetzt werden, dass zum einen keine Fehlerhaften Signale auf den Leitungen vorkommen. Diese Fälle wurden schon in den Punkten "Querverweis" und "Querverweis" gelöst, sodass diese nicht auftreten. Zum Schutz der GPIO Pins sind noch Gatewiderstände notwendig. Diese variieren je nach Gatekapazität von MOSFET zu MOSFET. Um das Gate auf LOW zu halten, wenn der GPIO PIN noch kein Signal sendet wurde auch am Gate des Mosfets ein 100kOhm Gate-Pulldown Widerstand eingebaut. Zuätzlich sollte die Matrix aber auch so umgesetzt werden, dass der Spieler auf dem Brett die Figuren bewegen kann, ohne darauf achten zu müssen, große Bewegungen um Felder zu machen, welche er nicht bespielen will.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

#### 3.1.2.4 Prototypenaufbau

Bei der Erstellung des Prototypen in Form einer 2x2 Matrix wurden als MOSFETs 2 IRLZ44 N-Kanal verwendet, da diese bereits zur Verfügung standen. Diese sind jedoch nicht perfekt da bei einer Gate-Spannung von 3,3V Der $R_{DSon}$ noch nicht minimal klein ist, allerdings ist es für die Testung ausreichend und durch die geringe Last von maximal ca. 30mW ist Wärmeentstehung durch den Widerstand auch nicht problematisch.

### 3.1.3 Platinendesign

### 3.1.4 Gehäusefertigung

### 3.1.5 Stromversorgung

## 3.2 Software

### 3.2.1 Magneterkennung

### 3.2.2 Matrixauslesung

### 3.2.3 Entwicklung eines Menüs

### 3.2.4 Spiellogik

#### 3.2.4.1 Das Einzelspiel (Spieler gegen Computer)

#### 3.2.4.2 Der Mehrspieler (Spieler gegen Spieler)

### 3.2.5 Einbindung des Spielererkennung

## 3.3 Webinterface

### 3.3.1 Website

### 3.3.2 Datenbank

### 3.3.3 Server

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

# 4 Latex-Beispiele

## 4.1 Auflistungen

- *Kursiv* Text 1
- **Fett**
- TT

Dasselbe durchnummeriert:

1. *Kursiv* Text 1
2. **Fett**
3. TT

## 4.2 Tabellen

Eine Tabelle mit Testdaten:

| position | mean | median | sd | min | max |
|---|---|---|---|---|---|
| **6** | 6.89 | 5.61 | 7.29 | 0.31 | 160.12 |
| **9** | 5.35 | 4.39 | 4.94 | 0.18 | 76.40 |
| **12** | 8.70 | 6.96 | 10.72 | 0.15 | 239.88 |
| **13** | 9.01 | 7.54 | 7.60 | 0.15 | 138.86 |
| **15** | 8.18 | 6.99 | 6.86 | 0.16 | 117.26 |
| **16** | 5.26 | 4.42 | 4.99 | 0.08 | 110.21 |
| **17** | 5.87 | 4.79 | 6.13 | 0.15 | 98.88 |
| **36** | 8.21 | 6.72 | 7.58 | 1.36 | 122.35 |
| **42** | 6.77 | 5.93 | 6.98 | 1.72 | 123.72 |
| **43** | 6.27 | 5.53 | 3.21 | 0.57 | 35.69 |

Tabelle 4.1: Eine Tabelle mit Testdaten

Sprachen wie z.B. **R** können Latex-Tabellen exportieren, sie müssen also nicht immer so aufwändig formatiert werden.

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

## 4.3 Abbildungen

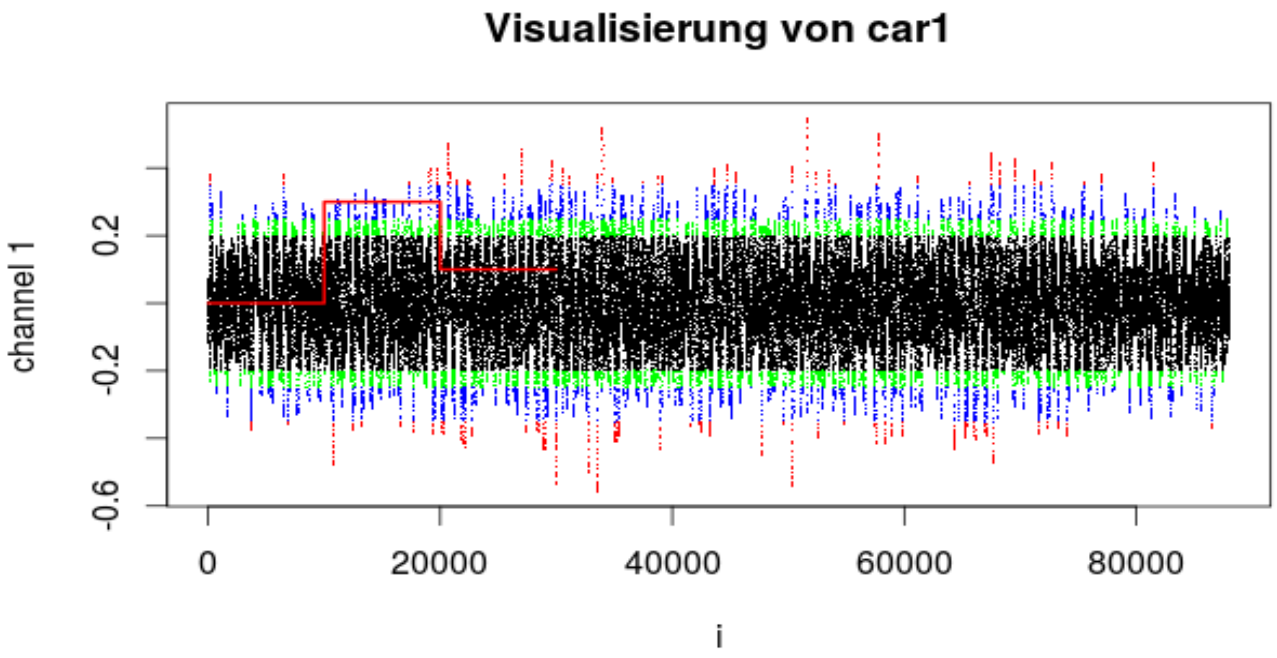


Abbildung 4.1: Ein Beispiel für ein Bild

## 4.4 Quellcode

Quellcode wird automatisch (mit der Möglichkeit die Sprache anzugeben) formatiert und in das Listings-Verzeichnis gegeben:

### 4.4.1 Java-Code

```
int i = 1;
float f = 2;
```

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

```
System.out.printf("Int-Z %d Float-Z: 52f",i ,f );
```

Listing 4.1: Java-Beispiel

### 4.4.2 Python-Code

```
#Hier ein kleines Beispiel in Python
lower = 0
upper = 10
for i in range(lower,upper):
    print(i)
```

Listing 4.2: Python-Beispiel

### 4.4.3 Lesen von Dateien

Es kann auch direkt von Dateien gelesen werden:

```
public class First {

  public static void main(String[] args) {
    for (int i = 0; i < 10; i++) {
      System.out.println(i);
    }
  }
}
```

Listing 4.3: Java-Beispiel von Datei

oder noch ein Beispiel in Python:

```
'''
Just a test program for latex
Markus
June 2024
'''

```

Maximilian Koch
Christoph Schwnaninger
Christina Seidner

```
a = int(input("pls enter a number ->"))
b = int(input("pls enter a number ->"))
print(f"{a}+{b}={a+b}")
```

Listing 4.4: Python-Beispiel von Datei

## 4.5 Referenzen

Beispiele für die Verwendung von Referenzen:

- Wie in Tabelle 4.1 auf Seite 24 ersichtlich...
- Wir sind im Kapitel 4
- In Zeile 2 im Listing 4.3

## 4.6 Zitate

Hier das Zitat eines Buches: Couper (2001).
Noch ein Zitat einer Zeitschrift: Reips (2002). Wird alles automatisch mit bibtex erledigt.

# Appendix

# Tabellenverzeichnis

3.1 Bewertung der Hall-Sensoren zur Figurenerkennung . . . . . 14
3.2 Bewertung von NFC/RFID zur Figurenerkennung . . . . . 15
3.3 Bewertung optischer Sensoren zur Figurenerkennung . . . . . 16

4.1 Eine Tabelle mit Testdaten . . . . . 24

# Abbildungsverzeichnis

3.1 Nahaufnahme des A3144 . . . . . 17
3.2 Verschaltung des A3144 . . . . . 17
3.3 Einzelbeschaltung . . . . . 19

4.1 Ein Beispiel für ein Bild . . . . . 25

# Listings

4.1 Java-Beispiel . . . . . 25
4.2 Python-Beispiel . . . . . 26
4.3 Java-Beispiel von Datei . . . . . 26
4.4 Python-Beispiel von Datei . . . . . 26

# Literaturverzeichnis

Couper, M. P. (2001), 'Web Survey Research: Challenges and Opportunities', Proceedings of the Annual Meeting of the American Statistical Association.

Diekmann, A. (1999), *Empirische Sozialforschung. Grundlagen, Methoden, Anwendungen*, fifth edn, Rowohlts Enzyklopaedie, Reinbeck bei Hamburg.

Dillman, D. A., Tortora, R. & Bowker, D. (1998), Principles for Constructing Web Surveys, Technical report, SESRC.

Loc (2004), 'Corporate value statement'. Einstiegsseite zum Unternehmensleitbild.
**URL:** *http://www.lockheedmartin.com/wms/findPage.do*

Nachname, V. (2024), 'Titel der webseite', https://www.beispielseite.de. Zugriff am 15. November 2024.

Reips, U.-D. (2002), 'Standards for Internet-Based Experimenting', *Experimental Psychology* **1**(4), 243–256.

*Titel der Website* (n.d.), Website. (Abgerufen am: 2017-07-11).
**URL:** *http://www.musterseite.de*